

Efficient Provenance Management via Clustering and Hybrid Storage in Big Data Environments

Die Hu[✉], Dan Feng, *Member, IEEE*, Yulai Xie[✉], *Member, IEEE*, Gongming Xu[✉], Xinrui Gu, and Darrell Long[✉], *Fellow, IEEE*

Abstract—Provenance is a type of metadata that records the creation and transformation of data objects. It has been applied to a wide variety of areas such as security, search, and experimental documentation. However, provenance usually has a vast amount of data with its rapid growth rate which hinders the effective extraction and application of provenance. This paper proposes an efficient provenance management system via clustering and hybrid storage. Specifically, we propose a Provenance-Based Label Propagation Algorithm which is able to regularize and cluster a large number of irregular provenance. Then, we use separate physical storage mediums, such as SSD and HDD, to store hot and cold data separately, and implement a hot/cold scheduling scheme which can update and schedule data between them automatically. Besides, we implement a feedback mechanism which can locate and compress the rarely used cold data according to the query request. The experimental test shows that the system can significantly improve provenance query performance with a small run-time overhead.

Index Terms—Big data, provenance management, clustering, hybrid storage, compress

1 INTRODUCTION

PROVENANCE is a kind of metadata that records the creation and transformation of data objects. Provenance increases the value of original data and has been typically used in security [1], search [2] and experimental documentation.

However, with the increment of user operations and system running time, the volume of provenance data is also growing rapidly. The provenance information of a data object is usually much larger than the data itself. In many cases, the provenance of a data object may even be more than ten times the size of the object itself. For example, in an online database MiMI [3] for storing protein information, the original data is 270 MB, but its provenance is 6 GB. Rapid growth rate and huge data volume makes it difficult to effectively extract and utilize provenance data.

A large number of studies [1], [4], [5], [6], [7], [8], [9], [10], [11] focus on provenance compression. For instance, Chapman et al. [5] proposed a series of decomposition and inheritance methods for compressing provenance. They reduced the provenance information by compressing common

subgraphs between different provenance graphs. Ma et al. [1] proposed ProTracer, a lightweight provenance tracing system which can process events through a concurrent userspace daemon. It can filter the redundant provenance through online analysis without affecting the accuracy of provenance analysis. These methods can effectively reduce the duplicate provenance data. However, this kind of data compression does not fully exploit the layout and the usage characteristics of the provenance. There are also researches on provenance query [2], [12], [13] based on efficient provenance management. In the case of the massive provenance environment, these management methods are not efficient enough. Typically, they do not exploit the relevance of the provenance nodes. Provenance data of the same node are still scattered in different provenance log files and the provenance data generated by different applications are indiscriminately managed. This has a negative impact on query performance.

Although there are a variety of provenance collection systems [14], [15], [16], [17], [18], [19], [20], they cannot manage the provenance efficiently in terms of the provenance storage and use. These systems: 1) do not exploit the characteristics of the provenance locality (e.g., provenance belonging to an event should be kept together). 2) do not store data based on provenance access characteristics such as hot and cold. 3) do not optimize the storage strategy based on the long-term usage of provenance.

This paper proposes an efficient provenance management method based on clustering and hybrid storage.

First, it can automatically perceive provenance features and cluster provenance data. Typically, we assign a value H to each node which indicates the importance of the node. The nodes with a big H ($>$ threshold T_H) are considered as *key nodes* and will get labels. We use the *key nodes* as the label

• D. Hu, D. Feng, and X. Gu are with the Wuhan National Laboratory for Optoelectronics, School of Computer, Key Laboratory of Information Storage System, Ministry of Education of China, Huazhong University of Science and Technology, Wuhan 430074, P.R. China. E-mail: {hudie, dfeng}@hust.edu.cn, guxinruigxr@163.com.

• Y. Xie and G. Xu are with the Wuhan National Laboratory for Optoelectronics, School of Computer, Huazhong University of Science and Technology, Wuhan 430074, P.R. China. E-mail: ylxie@hust.edu.cn, xugongming38@gmail.com.

• D. Long is with the Jack Baskin School of Engineering, University of California, Santa Cruz, CA 95064. E-mail: darrell@ucsc.edu.

Manuscript received 24 July 2018; revised 12 Dec. 2018; accepted 19 Mar. 2019. Date of publication 25 Mar. 2019; date of current version 25 Nov. 2020. (Corresponding authors: Yulai Xie and Dan Feng.)

Recommended for acceptance by D. Puthal.

Digital Object Identifier no. 10.1109/TBDDATA.2019.2907116

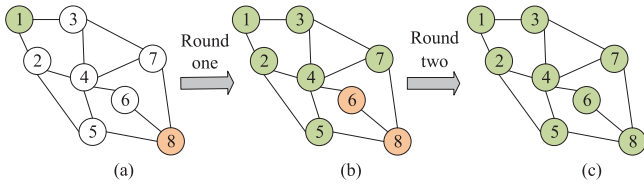


Fig. 1. Provenance label propagation using LPA (The shaded nodes represent the labeled nodes; The blank nodes represent unlabeled nodes; The numbers in them represent their *pnode numbers*).

propagation center to label nodes that are closely related to them. Then, we cluster nodes with the same label.

Second, as SSD (i.e., Solid-State Disk) has both a high speed and a high price and HDD (i.e., Hard Disk Driver) is low-cost and high-capacity, it dynamically perceives the data usage during the provenance query process to store hot data which is frequently accessed to SSD and store cold data to HDD. It also implements a strategy for scheduling hot and cold data between them.

In addition, it compresses the provenance information which is rarely or never used for a long time according to the monitoring of provenance usage.

The contributions of this paper are as follows:

- We implement a novel method called Provenance Label Propagation Algorithm to cluster the provenance data and store highly relevant provenance nodes into the same cluster as much as possible. This significantly improves the provenance query performance.
- We differentiate between hot and cold provenance according to the query request and implement a hybrid storage architecture which can store and schedule hot and cold data in SSD and HDD respectively.
- To further reduce the storage overhead, we implement a feedback strategy to compress the provenance information that has not been used for a long time.
- We implement and evaluate our system with various provenance workloads. The experimental results show that provenance clustering and hybrid storage significantly improve the provenance query performance with both a small runtime overhead and a small space overhead.

2 BACKGROUND AND MOTIVATION

2.1 Provenance System

Provenance information forms a Directed Acyclic Graph (DAG) [21]. The nodes in DAG represent the objects and the edges represent the dependencies between the objects. Currently, various academic institutions around the world have designed a variety of systems that collect provenance such as PASS [15], LinFS [22], SPADE [23], Story Book [24], TREC [25], CamFlow [26], etc. A common feature of these systems is that they collect dependencies between files and processes by monitoring system calls.

For instance, both PASS and SPADE are system-level provenance systems which collect provenance automatically by intercepting applications' read and write I/O operations and provide basic provenance generation, processing, and query functions. They mainly collect the provenance of the

following three kinds of objects: file object, process object, and network connection object. For the file object, its attributes contain the specific information of the file itself, such as the file name, file storage location, and the file's inode number. For the process object, its attributes mainly include the process name, process PID number, and environment variables. And for the network connection object, it is used to record the transmission of data on the network. Both PASS and SPADE record the dependencies between objects and assign a unique identifier to each object.

2.2 Label Propagation Algorithm

The Label Propagation Algorithm (LPA) [27] is a typical semi-supervised machine learning algorithm. The description of the LPA algorithm is as follows.

For each node p in the graph ($p \in G(p)$, $G(p)$ is a collection of all nodes), assign a unique label L_p to represent the community in which the node is located.

Step 1. Traverse all nodes. For each node, obtain the labels of its neighbors and find the label with the biggest number, then replace the original label of the node with this label. If there is more than one label with the biggest number, select a label randomly to replace the original label of this node. The mathematical expression is

$$\forall p \in G(p), L_p = \arg \max_{J_i} \sum n(p, J_i). \quad (1)$$

$n(p, J_i)$ denotes p 's neighbor node whose label is J_i , and $\arg \max_{J_i}$ denotes the label J_i which has the biggest number in p 's neighbors.

Step 2. Cycle step 1 until the labels of all nodes no longer change.

The time complexity of LPA is: $O(k * m)$, where k represents the number of iterations, and m represents the number of edges.

There are three problems if LPA is applied directly to the clustering of provenance nodes.

First, LPA is a semi-supervised learning method, which can process a set of nodes that already have an original label before the algorithm runs. Unfortunately, most of the provenance datasets are unlabeled at the initial time. We need to design a method to select some of the nodes before clustering and manually label them.

Second, many provenance nodes depend on shared system files. For instance, many processes use shared header files or library files as input. Each of the process node and the shared system files belong to a cluster that shows how this process is executed. Thus these system files will be relevant to multiple clusters. However, the information of each system file will be only stored once in order to save the storage space. Thus the node that represents the system file can only be included in a single cluster. However, LPA makes a random selection when there is more than one label with the biggest number in its neighbors causing that the shared node may not be divided into the most relevant cluster.

Third, due to the uncertainty of the propagation sequence, inaccurate clustering will be caused. The extreme case where all nodes are classified into the same cluster can happen. For example, in Fig. 1a represents the initial state of nodes 1-8 in which node 1 and node 8 are labeled with their identifiers and other nodes do not have initial labels. We carry out the

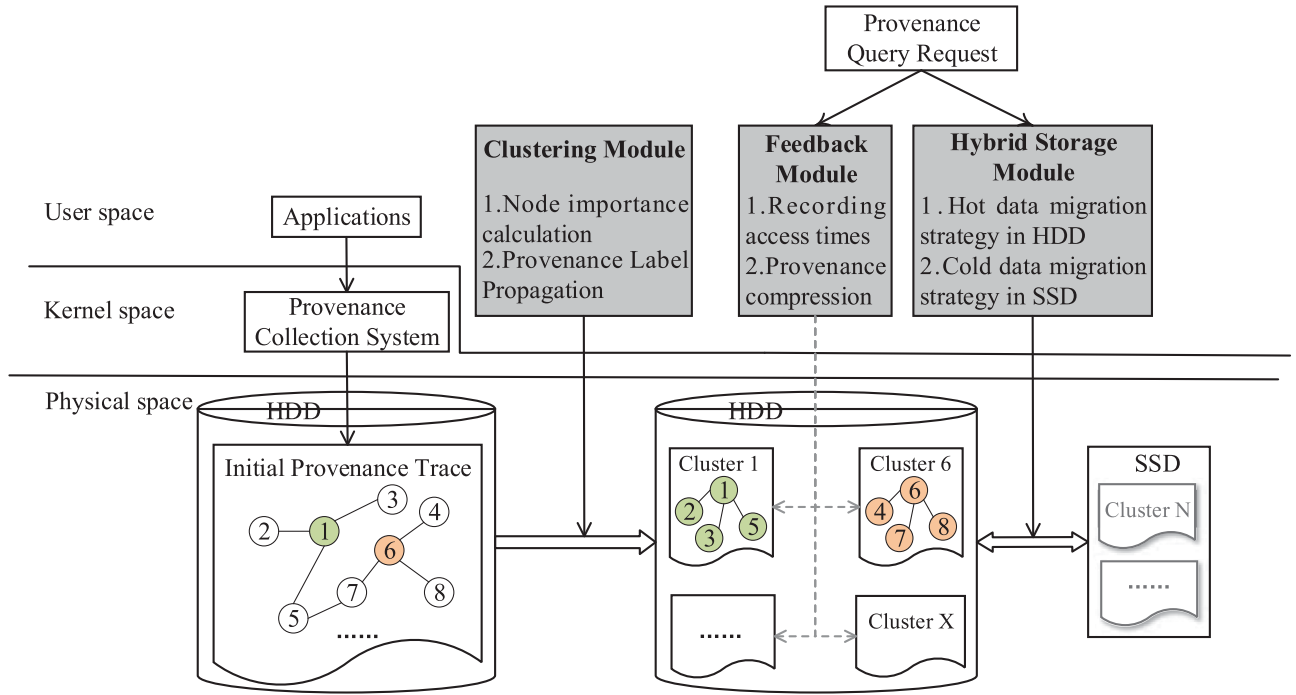


Fig. 2. System framework.

first label propagation from node 1 to node 8. Nodes 2, 3, 4, 5, and 7 will select node 1's label as their own labels in turn. This is because the label of node 1 appears the most in their adjacent nodes. Node 6 chooses node 8's label as its label at the beginning of the propagation. As the second label propagation starts, the label of node 8 will be covered by the label of node 1 because node 8's neighbors nodes 5 and 7 are both with label 1. The label of node 6 will be covered by the label of node 1 for the same reason. It can be seen that node 8 has no opportunity to spread its label because of the propagation sequence. Finally, the whole graph is labeled with label 1. This means that there is no division for the graph.

2.3 HDD and SSD

In recent years, the main storage device in the computer system is the hard disk drive (HDD), which is a non-volatile, low-cost, high-capacity storage device. Due to the mechanical components of the hard disk, its performance bottleneck is mainly caused by the I/O access. Besides, the performance gap between the storage system and the CPU is also increasing [28].

Solid state disk (SSD) is a data storage device which uses NAND flash to store data persistently [29]. Compared with HDD, SSD has better read/write performance, but lower capacity and higher price [30]. Therefore, SSD can be integrated into existing HDD-based storage hierarchies to allow storage systems to achieve higher access performance.

We propose to use hybrid storage technology to manage the provenance data. We can achieve better access performance by classifying data according to data significance or making reasonable scheduling for data processing [31]. We consider storing provenance data that has not been used for a long period in HDD to reduce storage space costs and migrate frequently accessed provenance data dynamically to SSD. This can improve the query performance of provenance as much as possible.

Using two kinds of devices to form hybrid storage is a new attempt for provenance data storage. It can fully combine the advantages of HDD and SSD, and effectively improve the management performance of provenance data.

2.4 Application Scenarios for Provenance Clustering

Provenance clustering has many applications. In the field of forensic analysis, in order to identify the source of system intrusion, it is necessary to traverse the provenance graph of the corrupted file or process. If the provenance of the same file or process can be clustered in the storage layout, the provenance query for forensic analysis can be improved.

In the field of public safety, the relationship between people and events in public safety events can be described by provenance graphs. When a public safety incident is queried, it is likely that the event data is stored with other unrelated events or people information. By clustering and storing the related event data of a provenance graph together, the query and analysis time can be greatly reduced.

The scientists reproduce the experiment by recording the necessary details of the experiment, such as the parameters used in the data set, the intermediate steps for generating the data set, etc. However, multiple experimental records/logs are usually stored in chronological order, which is not conducive to reproduce the experiment. By clustering the provenance records for each experiment and storing them in units, scientists can quickly and accurately reproduce the experiment.

3 DESIGN AND IMPLEMENTATION

3.1 Overall System Design

The overall architecture of the system is shown in Fig. 2. The provenance management system consists of three modules: a clustering module, a hybrid storage module, and a feedback module.

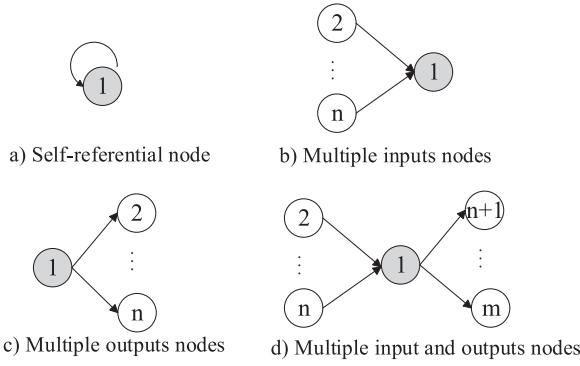


Fig. 3. Node types in provenance graphs.

The *clustering module* aims to store the provenance node with strong relevance together using Provenance Label Propagation Algorithm. It first calculates the importance degree of the provenance nodes, then labels each high importance node a globally unique identifier (called *pnode number*) and make it as the center for label propagation. Finally, the provenance nodes with strong relevance will be labeled with the same identifiers and extracted into the same cluster.

The *hybrid storage module* enables separate storage of cold data and hot data using two kinds of storage devices, HDD and SSD. The provenance processed by the clustering module is first stored in the HDD indiscriminately. When a query request arrives, the result is sent back and the related nodes are swapped out to the SSD. Besides, a list which records the query history is maintained. After the SSD is full, the data in the SSD would be swapped out and sent to the HDD using the LRU algorithm.

The *feedback module* monitors cold data in the HDD continuously. The provenance that has not been used for a long time will be compressed to save the space overhead.

3.2 Provenance Clustering

Provenance can reflect the historical change of data objects. The provenance collection system generates provenance by intercepting system calls and stores it in chronological order. This may cause that similar provenance nodes are scattered in different logs. This paper proposes PLPA, a Provenance Label Propagation Algorithm which clusters the provenance based on *key nodes* and stores highly relevant provenance nodes into the same cluster to improve the query performance.

There are two steps to implement *key node* based clustering: node importance calculation and provenance label propagation. First, we assign a H value to each node which indicates the importance of the node, and consider the nodes whose value H exceeds T_H as the *key node*, and label these nodes with their *pnode numbers*. This solves the problem that provenance nodes have no label at the initial time. Then we use these *key nodes* as the label propagation center to label the nodes which are directly connected to them. The label propagation will be performed in descending order of H values which ensures that nodes with high H have priority to propagate and thus all nodes will be divided into a more relevant cluster. In this way, we can store nodes with the same label into the same provenance cluster.

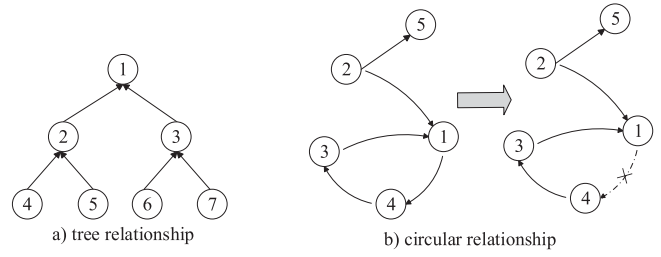


Fig. 4. Dependencies between nodes.

3.2.1 Node Importance Calculation

The importance of the nodes are calculated by their dependencies. The value H of node p is calculated as follows:

$$H(p) = H_0 + \sum_{p_i \in M_p} H(p_i). \quad (2)$$

H_0 represents the initial importance degree of a node. M_p represents a collection of nodes which point to node p .

Algorithm 1. Calculation of Value H of Node p

Function name: computeH(p)
Input: p
Output: $p.H$

```

1: if  $p.input$  is null then
2:    $p.H = 1$ ;
3:   return  $p.H$ ;
4: end if
5: if  $p.input$  is  $p$  then
6:    $p.H = 0$ ;
7:   return  $p.H$ ;
8: end if
9:  $p.H = 1$ ;
10: for  $q$  in  $p.input$  do
11:    $p.H += \text{computeH}(q)$ ;
12: end for
13: return  $p.H$ ;
```

As shown in Fig. 3, there are four types of nodes in the existing provenance graphs: self-referential node, the node with multiple inputs, the node with multiple outputs and the node with both multiple inputs and multiple outputs. There will be two kinds of dependencies abstracted from these four types of nodes: tree relationships, and circular relationships, as shown in Fig. 4.

The *self-referential node* is shown in Fig. 3a. This type of node is self-dependent. It is usually an intermediate node that is generated in the provenance collection process without any attributes. So we define Rule 1 and Rule 2.

Rule 1. If a node p points to itself and no other node points to it, then its $H_0 = 0$;

Rule 2. If there is at least one node pointing to p which does not point to itself, then for p , its $H_0 = 1$;

The value H of the *tree-dependent node* (as shown in Fig. 4a) can be deduced through Rule 1, Rule 2 and Formula 2.

The *circular relationship* in the provenance graph is shown in Fig. 4b. Nodes 1, 3, and 4 form a circular relationship. If we calculate the value H of them according to Formula 2, we will get into an endless loop. So we define rule 3.

Rule 3. If node p is in a circular relationship and is the terminal node (e.g., node 1 in Fig. 4b) of the cycle, the edge in

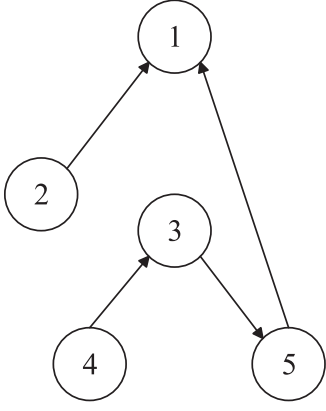


Fig. 5. Provenance nodes relationships.

the cycle starting with p will be broken. Then we calculate the value H of nodes in the new relationship using Rule 1, Rule 2 and Formula 2.

We use a recursive algorithm to calculate value H . The algorithm process is shown in Algorithm 1, in which, $p.H$ is the value H of node p and $p.input$ is the node which points to node p . When $p.input$ is null, p is a root node and its H is set to 1. When $p.input$ is p , p is a self-referential node and its H is set to 0. When there are multiple nodes pointing to p , $p.H$ is the sum of its initial value that is set to 1 and the H of $p.input$.

Before the label propagation process, a part of nodes must be labeled. We set a threshold T_H and consider the node whose value H exceeds T_H as the *keynode*. These *keynodes* take the *pnode number* as their label, which provides a basis for the label propagation.

3.2.2 Provenance Label Propagation Algorithm

To avoid the defects of LPA, we propose PLPA, a Provenance Label Propagation Algorithm which is suitable for the provenance clustering.

First, PLPA is an active propagation process. PLPA regards the labeled node as the active object and allows it to propagate its labels actively to the unlabeled nodes directly connected to them. Comparatively, LPA regards the unlabeled node as the active object and allows it to find the label with the biggest number in its adjacent nodes. This is a passive propagation process and will cause that all nodes are classified into the same cluster for provenance clustering.

Second, PLPA conducts provenance label propagation in descending order of importance degree to ensure that nodes with high importance can obtain the priority in label propagation.

The process of PLPA is as follows:

Step 1. Label the nodes whose value H is greater than the threshold T_H with their *pnode numbers*. Let p represent a node in the provenance graph ($p \in G(p)$, $G(p)$ is a collection of all nodes). L_p is the label of node p ;

Step 2. Sort all nodes in descending order of values H ;

Step 3. Traverse all nodes. If the current node p is labeled as L_p , find all nodes directly connected to it and label them. If the current node is an unlabeled node, skip it.

Step 4. Repeat step3 until the total number of unlabeled nodes is less than threshold X .

Algorithm 2 shows the pseudorandom code. In Algorithm 2, *nodes number* represents the number of

Original provenance nodes, and $nodes[i]$ represent the node whose *pnode number* is i in the trace. $nodes[i].H$ and $nodes[i].Label$ represent the H value and *Label* of $nodes[i]$, respectively. The $sort(all\ nodes, H)$ function sorts all nodes in descending order of H values, and the $getSRC(nodes[i])$ function gets all the nodes pointing to $nodes[i]$.

Algorithm 2. The Process of PLPA

Function name: PLPA
Input: *Original provenance nodes*
Output: *Clustered provenance nodes*

```

1: for i = 0; i < nodes number; i++ do
2:   if nodes[i].H == null then
3:     computeH(nodes[i]);
4:   end if
5:   if nodes[i].H > TH then
6:     nodes[i].Label = i;
7:   end if
8: end for
9: sort(all nodes, H);
10: for i = 0; i < nodes number; i++ do
11:   SRC[ ] = getSRC(nodes[i]);
12:   for j = 0; j < sizeof(SRC); j++ do
13:     if SRC[j].Label == null then
14:       SRC[j].Label = nodes[i].Label;
15:     end if
16:   end for
17: end for

```

Assuming that the number of nodes in the trace is V and the number of edges is E . The time complexity consists of three parts.

First, from Section 3.2.1, the H value is calculated by the number of edges that point to a node, and each edge is only used once. So it is only positively correlated with the number of edges and thus the time it takes to calculate the H value is $O(E)$;

Second, the sorting time, we use the method of quick sorting, so it is $O(V \log(V))$;

Third, from Section 3.2.2, each edge produces a propagation and does not have repeated propagation. So the process of label propagation is only related to the number of edges. Thus the time of label propagation is $O(E)$.

So the time complexity of the PLPA algorithm is

$$O(2 * E + V \log(V)). \quad (3)$$

Fig. 5 shows a provenance graph of several nodes, and Fig. 6 shows its label propagation process using PLPA in which nodes are arranged from left to right in descending order of H value. The shaded nodes represent the nodes that have been labeled. The blank nodes represent unlabeled nodes. The numbers in them indicate their *pnode numbers*. In the initial state, only the nodes whose value H is greater than the threshold T_H are labeled with their *pnode numbers*, such as node 1. In the *first round* of propagation, propagation starts from node 1 with the highest value H . Node 1 propagates its label to node 2 and node 5 which have direct relationships with node 1. When traversing to node 5 which is already labeled with the label of node 1, node 3 which is connected to node 5 will be labeled. It can be seen that node 4 fails to obtain a label during the first propagation round since node 3 is

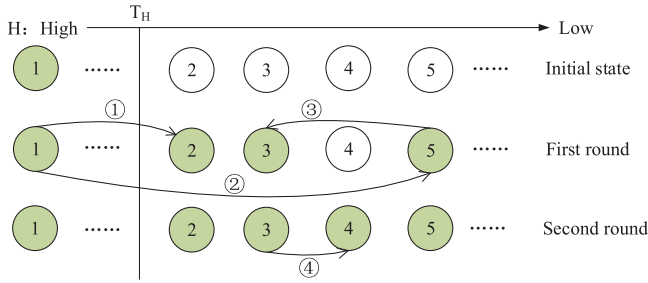


Fig. 6. Provenance label propagation (The shaded nodes represent the labeled nodes; The blank nodes represent unlabeled nodes; The numbers in them represent their *pnode numbers*).

unlabeled at the first time it is traversed. In the *second round* of propagation, the label of node 3 is propagated to node 4. Considering that there may be a small number of isolated nodes that cannot be propagated. Therefore, when the total number of unlabeled nodes is less than a predefined threshold X , the propagation process will stop.

3.3 Hybrid Storage

If provenance nodes are stored on the HDD indiscriminately, access delays will be caused in multiple lookups. For example, when analyzing a network intrusion path, a socket provenance node containing IP information is the starting point for all operations, and all path analysis starts from it. At this point, the node will be a *key node* and will be visited multiple times to become a hot data node. Considering that most of the provenance is cold data that is rarely accessed and only a small part of provenance is repeatedly accessed hot data, we design a hybrid storage module to store the cold and hot data separately. As SSD has a better read/write performance than HDD, we add an SSD to store the frequently accessed nodes and their associated nodes which may be accessed in a short time and use HDD to store the cold nodes. The hybrid storage architecture will make full use of SSD and HDD's advantages in cost, read/write mode, and capacity. Thus it will improve the query performance of provenance. The specific hybrid storage architecture is shown in Fig. 7.

The hot/cold scheduling module implements two functions. When the data in the HDD is frequently accessed, the hot data in the HDD is moved to SSD; when the SSD is full, the data in the SSD will be moved to HDD. In order to locate the least-recently-used provenance nodes quickly, we maintain an LRU queue to record the provenance nodes stored in the SSD and update the LRU queue after each movement.

3.3.1 Hot Data Migration Strategy in HDD

Provenance records the transformation of data objects. In the usual case, most of the provenance nodes are cold data, i.e., they are not accessed for a long time. Once a provenance node is accessed, it indicates that the importance of the node increases and it will be probably visited in the following period. So, we design the following strategy.

When receiving a query request of a provenance node, the system first performs a query in SSD. If the provenance is not found, it needs to be sought again in HDD. If it is found in HDD, the provenance node has changed from a cold node to a hot node. Then we copy the node to SSD and delete the copy in HDD.

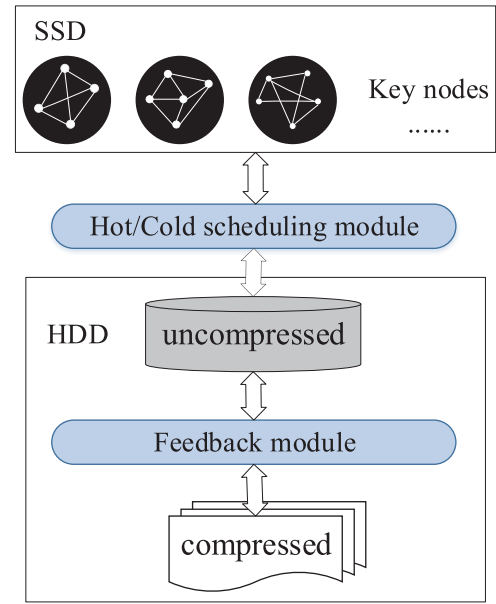


Fig. 7. Hybrid storage structure.

3.3.2 Cold Data Migration Strategy in SSD

As represented in Algorithm 3, we use ST_{max} and ST_{min} to represent the maximum and the minimum amount of provenance nodes that SSD can store respectively. $S_{current}$ indicates the used capacity of the SSD at the current time. When a new query request for node p arrives, if the SSD is full, the least-recently-used provenance nodes in the SSD are moved to HDD until the SSD usage is equal to ST_{min} which ensures that the most-recently-used hot nodes in the SSD will not be moved out and reduces movement times. The cold provenance will be moved to HDD in the condition that a new provenance node is written to SSD which is already full as shown in Algorithm 3.

Algorithm 3. Cold Data Migration Process

Function name: migrationToHDD (p)

Input: p

Output:

```

1: LRU.head ++;
2:  $S_{current} = S_{current} + \text{sizeof}(p)$ ;
3: if  $S_{current} > ST_{max}$  then
4:   for  $S_{current} > ST_{min}$  do
5:     SSDtoHDD(LRU.tail);
6:      $S_{current} = S_{current} - \text{sizeof}(LRU.tail)$ ;
7:     LRU.tail --;
8:   end for
9: end if

```

3.4 Feedback

As shown in Fig. 7, the feedback module analyzes the usage of cold data in the HDD and compresses the cold data that is not used for a long time to reduce the storage overhead.

Specifically, the feedback module records the node's query requests and compresses the long-term unused provenance nodes in HDD. As we can see from Section 3.2, the closely-related nodes have the same label and are stored in the same cluster. Therefore, we maintain a request time for each cluster.

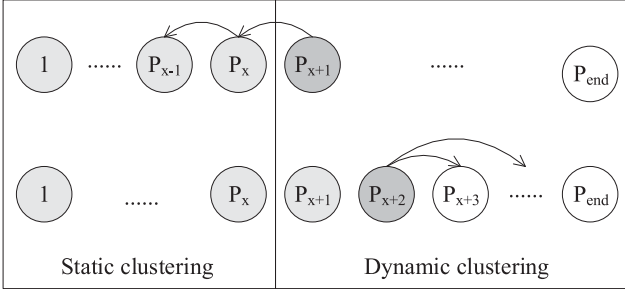


Fig. 8. Incremental clustering process.

As represented in Algorithm 4, R_i is the request time of cluster i . We define the feedback interval T_{feed} and perform the compression operation in every interval. Specifically, the feedback module scans the R_i of all clusters, then it compresses clusters whose R_i is smaller than R_X (A threshold that the number of times the cluster is accessed) and adds them to a list that indicates which clusters have been compressed.

Algorithm 4. Feedback Mechanism

```

Function name: feedback ()
1: if Sysruntime >  $T_{feed}$  then
2:   for Cluster $_i$  in Clusters do
3:     if  $R_i < R_X$  then
4:       compress(Cluster $_i$ );
5:       update(compressionlist);
6:     end if
7:      $R_i = 0$ ;
8:   end for
9: end if

```

3.5 Incremental Maintenance

We call the clustering process for the complete provenance information of a period as static clustering. In an actual provenance application, provenance may be generated indefinitely. The newly generated provenance node may be associated with the provenance nodes which have been clustered. In this case, labels of clustered provenance node may change. In order to test the clustering effect of the growth provenance, we designed an incremental maintenance module to implement the dynamic clustering.

We simulate the incremental clustering process using the existing provenance dataset and assume that its *pnode numbers* are from 1 to P_{end} . We select the nodes whose *pnode numbers* are from 1 to P_x ($P_x < P_{end}$) to perform a static clustering process (i.e., we first calculate the value H of the nodes, and then perform the provenance label propagation algorithm to label them), as shown in Fig. 8.

In the provenance incremental clustering process, we consider a node as an incremental unit. There are two kinds of cases that may happen. First, as we can see from Fig. 8, node P_{x+1} points to node P_x whose value H is determined in static clustering. In this case, we need to recalculate the value H of P_x and the chain pointed by P_x . If there are new nodes whose value H is bigger than threshold T_H , it is necessary to relabel all nodes and the clustering result may change. Second, node P_{x+2} points to the node whose *pnode numbers* is bigger than itself, we just need to calculate

TABLE 1
Provenance Datasets Parameters

Type	Application	Nodes	Edges	Size	Source
PASS	Blast	1668	6776	1.1 MB	[8]
	Postmark	2681	4740	695.2 KB	
	Distcc	2696	14397	884.3 KB	
	Chall3	3470	14758	1.8 MB	
	ProFTP	5998	7000	1.1 MB	[32]
	VSFTP	10776	14429	1.6 MB	
SPADE	Blast	415	488	52.2 kB	[33]
	Postmark	1623	1704	346.7 KB	
	Apache	240191	296417	27.3 MB	

the value H of P_{x+2} and label it. The whole process includes four steps.

Step 1. Read a new provenance node P_{cur} and calculate the value H of P_{cur} .

Step 2. Find the chain pointed by P_{cur} . If there is any node in the chain, recalculate the value H of the nodes in the chain and jump to Step 3. Otherwise, label node P_{cur} and jump to Step 4.

Step 3. Perform the label propagation to relabel nodes 1 to P_{cur} .

Step 4. Repeat Step 1 until $P_{cur} = P_{end}$.

In this way, we ensure that the nodes on the provenance chain get the latest value H and label.

4 EXPERIMENTAL EVALUATION

4.1 Experimental Setup

We perform all of the experiments on a machine with 8 Intel i7-6700 processors and 32 GB memory. It has 4 TB HDD and 256 GB SSD for storage. The host machine runs Windows 10.

4.2 Provenance Datasets

As shown in Table 1, we demonstrate our system with two types of provenance traces, which were generated by PASS [15] and SPADE [23], respectively.

There are six PASS traces and three SPADE traces. Specifically, Blast is a biological workload. Postmark simulates the small file read/write workload. Distcc is a distributed C/C++ compilation tool and the dataset is collected when it is compiled. Chall3 is collected when the workflow defined in the third provenance challenge is carried out. ProFTP and VSFTP are both FTP servers on Unix platforms and the datasets are collected when they are exploited by the malicious users. Apache is a web server and its provenance recodes the web page request and visit. Note that both Blast and Postmark have different formats in PASS trace and SPADE trace. The details (node number, edge number, size etc.) of the traces are shown in Table 1.

4.3 Query Performance of Cluster

In this section, we give the query performance of original, K-means, LPA and PLPA on each trace. For the K-means algorithm, we extracted four most useful attributes (ie, the type of node, the pid of node, the count of node's sources and the count of node's destinations) from the provenance traces to perform clustering.

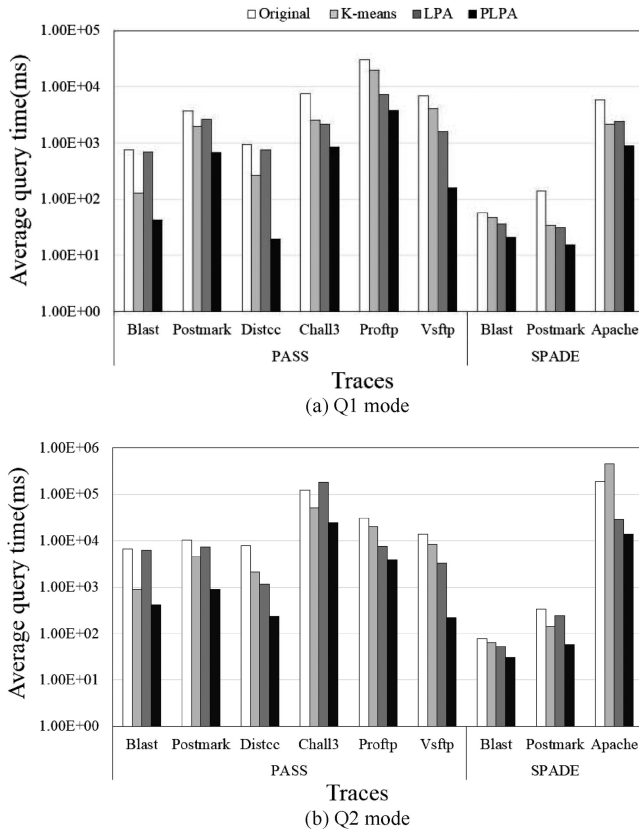


Fig. 9. Query performance of cluster on PASS and SPADE.

To verify the effectiveness of clustering, we use the following two query modes with different time complexity to test the system.

Q1: For a node, query its attribute information and its ancestor nodes that it directly depends on.

Q2: Query the whole ancestor chain of the node.

Fig. 9 shows the query performance of PASS and SPADE traces for different clustering method on Q1 mode and Q2 mode. In the Q1 mode, the average query time of PLPA clustered provenance reduces by 56-98 percent compared to the original data, reduces by 56-93 percent compared to the K-means clustering, and reduces by 35-63 percent compared to the LPA clustering. In the Q2 mode, the average query time of PLPA clustered provenance reduces by 38-90 percent compared to the original data, reduces by 51-97 percent compared

TABLE 2
Time Overhead of Clustering on PASS and SPADE

Type	Application	Average clustering time (ms)		
		K-means	LPA	PLPA
PASS	Blast	708.633	0.085	0.153
	Postmark	71.242	0.054	0.142
	Distcc	263.724	0.154	0.261
	Chall3	40.922	0.148	0.214
	ProFTP	63.188	0.092	0.099
	VsFTP	183.370	0.209	0.194
SPADE	Blast	0.880	0.005	0.014
	Postmark	0.143	0.001	0.002
	Apache	85.595	0.125	0.182

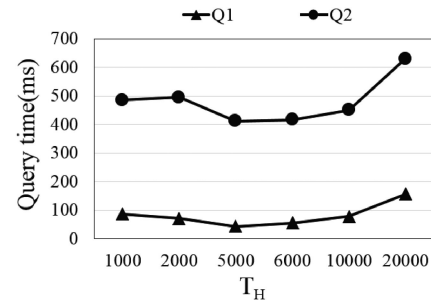


Fig. 10. Query performance on Blast PASS trace at different thresholds.

to the K-means clustering, and reduces by 24-52 percent compared to the LPA clustering.

After clustering, the original provenance is divided into small clusters. Therefore, when performing Q1 and Q2 mode queries, the system does not need to completely prefetch the entire original provenance. Instead, all the information of a provenance node is stored in a small cluster, and the traversal is performed in units of small clusters. This greatly reduces the query time.

However, after clustering by LPA, we find that there is only one node in many provenance clusters, and this means that a single node is treated as a cluster. This causes the clusters searching on the storage device to be performed multiple times when the query is executed. Sometimes clustered provenance by LPA takes more query time than the original provenance. PLPA can make good use of the relationship in the edge information of the provenance trace, so the provenance nodes in the same cluster are highly relevant and thus have a good clustering effect. K-means is suitable for clustering of discrete points in high dimensional space, and cannot utilize the edge information of provenance well. Thus it may make relevant nodes clustered into different clusters, causing the clustering effect is poor.

Table 2 shows the average clustering time of PASS and SPADE traces. The average clustering time of each trace is less than 1 ms, both on LPA and PLPA. This is much smaller than the query time. Due to the high time complexity of the K-means algorithm, its average clustering time is 2 - 4 orders of magnitude higher than the other two algorithms. The time overhead of clustering on PLPA is negligible compared with the time reduced on the query. This proves that it is effective to cluster provenance using PLPA.

4.4 Optimal Threshold Selection

The H value is calculated by the edge relationship between the nodes. Since different traces have the different number of nodes and edges, their ranges of H values are different from each other.

We use the Blast PASS trace as an example. The H value of the trace ranges from 0 to 54626. We select 1000, 2000, 5000, 6000, 10000, and 20000 as the T_H to perform the clustering process. Fig. 10 shows the query performance of Blast PASS trace on different T_H . The lower line in the figure represents the average query time on the Q1 query mode, and the upper line represents the average query time on the Q2 query mode. As can be seen from the figure, the query performance is best when the threshold is 5000. So 5000 is the optimal T_H of the Blast PASS trace.

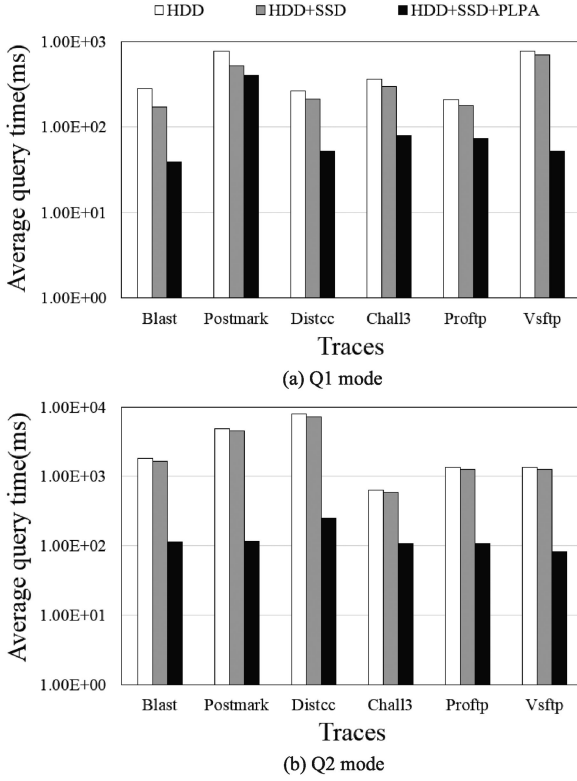


Fig. 11. Query performance of provenance traces with and without hybrid storage on PASS.

4.5 Query Performance of Hybrid Storage

To verify the effectiveness of hybrid storage, we perform provenance query on HDD and HDD+SSD respectively. In addition, for the HDD+SSD case, we also perform a query experiment on the provenance that has been clustered using PLPA. Fig. 11 shows the query performance on PASS provenance with and without hybrid storage. Compared to HDD case, the HDD+SSD case reduces query time by 6-39 percent. This is because, in the HDD+SSD case, most of the provenance is gotten from SSD, thus has a better performance than the case that only gets data from HDD. Under the condition of hybrid storage, the query efficiency of clustered data is increased by 98 percent compared with that of un-clustered. Because large provenance trace is split into small clusters after clustering, cluster's prefetching takes less time than the original trace.

4.6 Query Performance of Feedback

Fig. 12 shows the query performance of the Q1 mode and Q2 mode respectively. Despite the query performance of the system with the feedback module will decrease because there is a decompression process when the compressed cold node is accessed, the query performance is still significantly improved compared to the original data without clustering and hybrid storage. Specifically, the query time reduces by 28-72 percent and 32-77 percent on Q1 and Q2, respectively.

The feedback module enables compression of clusters that have not been queried for a long time on the HDD, thereby reducing storage overhead. The storage space before and after compression is shown in Fig. 13. It can be seen that the compression can significantly reduce 77-90 percent storage space for the six datasets.

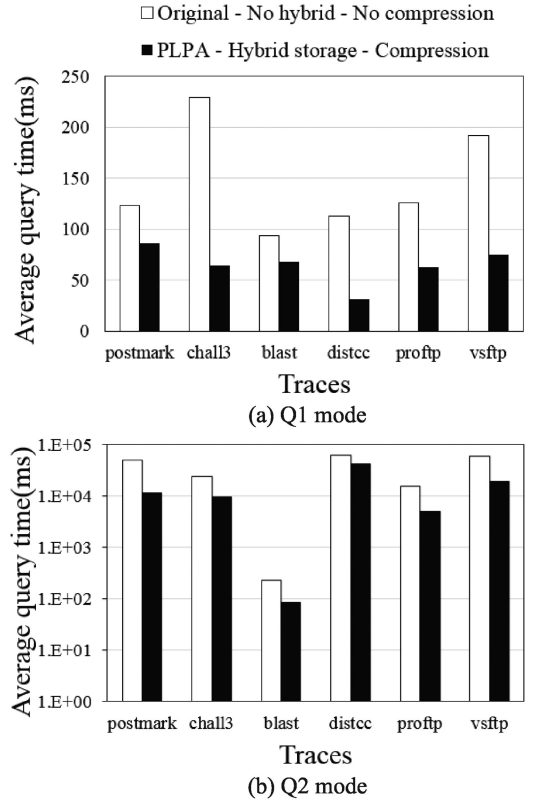


Fig. 12. Query performance of compressed and raw provenance on PASS traces. (Original - No Hybrid - No compression indicates the query time of original provenance without hybrid storage and compression. PLPA - Hybrid - Compression indicates the query time of provenance using PLPA clustered with hybrid storage and compression.).

4.7 Dynamic Clustering Overhead

We conducted dynamic label propagation experiments on the blast dataset which has 1668 nodes. In the blast data set, the nodes whose pnodes are between 1 to 1568 are selected for static label propagation. The last 100 nodes are used for dynamic label propagation. When the dynamic propagation is over, the labels of all nodes in the data set have the same labels as they have in the completely static propagation. This shows that the provenance label propagation is efficient on the incremental provenance. The dynamic propagation time for these 100 nodes is shown in Fig. 14. The time overhead is mainly caused by label propagation. We will

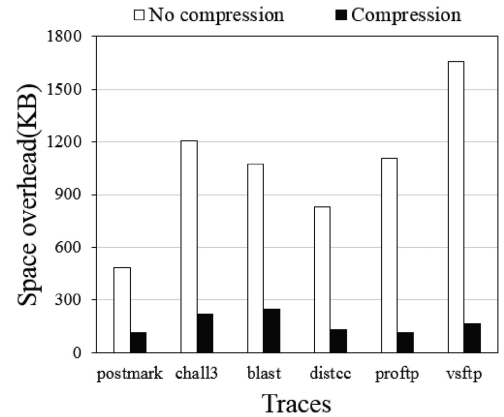


Fig. 13. Space overhead of compressed and raw provenance on PASS provenance traces.

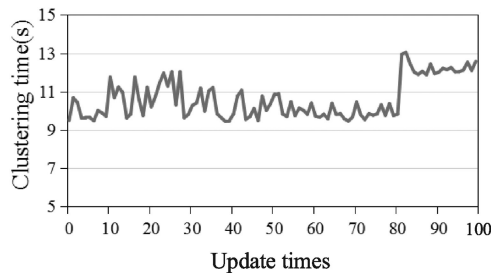


Fig. 14. Dynamic clustering overhead.

not perform label propagation every time a node is generated but we perform it when the system is idle. This can significantly reduce the impact on provenance query.

5 RELATED WORK

5.1 Provenance Management

A number of provenance collection systems [21], [22], [23], [24], [25], [26] have been developed. There are some studies on provenance management in specific areas.

For provenance compression, Xie et al. [7] explored the similarity and locality of provenance nodes and designed a hybrid compression algorithm that combines WEB compression with dictionary encoding. Zhang et al. [10] proposed CPR and PCAR which can aggregate the same type of events with the same attribute and reduce data effectively. Ma et al. [1] proposed a lightweight provenance tracing system ProTracer, which can process events through a complicated concurrent userspace daemon. It can filter the provenance of redundant events through online analysis and get a less space consumption without affecting the accuracy of provenance analysis. Lee et al. proposed LogGC [4] and BEEP [11]. The former is an audit log system with garbage collection capability. The latter is a highly accurate attack provenance tracing technique enabled by a novel selective fine-grained logging method which avoids the dependence explosion problem with regular audit logs. These methods can effectively reduce the duplicate provenance data. However, this kind of data compression does not fully exploit the layout and the usage characteristics of the provenance.

For provenance query, Macko et al. [12] proposed a new metric called ancestor centrality (AC) and a threshold detection algorithm to realize a local clustering for directed acyclic provenance graphs. Liu et al. [2] designed a scheme called P-index which utilizes data provenance to implement a high-performance metadata search for cloud storage and solves the query problem of cold data. Bao et al. [13] proposed to use the tree structure to store provenance and reduce the storage space of redundant information by using some of the nodes in the tree structure, and then proposed a dynamic programming solution for tree-structured queries without additional overhead.

5.2 Hybrid Storage

Feng et al. [29] proposed HDStore with a vertical architecture. It saves the journal file on the SSD to enable efficient read and write and stores segment files on the HDD to provide massive storage. In a horizontal hybrid storage architecture [34], [35], [36], [37], SSD is used to store small, frequently accessed hot data, while disks store large cold data. Jiang et al. [38]

proposed HiCH, which divides different storage devices into different buckets and improves the performance of the storage system. However, this scheme is only for replica storage strategies and has poor adaptability.

6 CONCLUSION

This article describes the design and implementation of a provenance management system based on event clustering and hybrid storage. First, the system enables efficient provenance clustering by improving the traditional label propagation algorithm. Second, by combining the advantages of HDD and SSD, we use hybrid storage strategy to implement real-time provenance queries. Finally, we implement a feedback mechanism to compress the provenance information with only a small runtime overhead. The experimental evaluation demonstrates that this system can effectively reduce storage costs and query time of provenance.

ACKNOWLEDGMENTS

This work was supported in part by the National Science Foundation of China under Grant No. U1705261, 61402189, 61821003 and 61772222, CCF-NSFOCUS KunPeng research fund, Wuhan Application Basic Research Program under Grant No. 2017010201010104, and Hubei Natural Science and Technology Foundation under Grant No. 2017CFB304.

REFERENCES

- [1] S. Ma, X. Zhang, and D. Xu, "ProTracer: Towards practical provenance tracing by alternating between logging and tainting," in *Proc. 23rd Annu. Netw. Distrib. Syst. Secur. Symp.*, 2016, pp.1–15.
- [2] J. Liu, D. Feng, Y. Hua, B. Peng, P. Zuo, and Y. Sun, "P-index: An efficient searchable metadata indexing scheme based on data provenance in cold storage," in *Proc. Int. Conf. Algorithms Archit. Parallel Process.*, 2015, pp. 597–611.
- [3] M. Jayapandian, A. Chapman, V. G. Tarcea, C. Yu, A. Elkiss, A. Ianni, B. Liu, A. Nandi, C. Santos, P. Andrews, et al., "Michigan molecular interactions (MiMI): Putting the jigsaw puzzle together," *Nucleic Acids Res.*, vol. 35, pp. D566–D571, 2006.
- [4] K. H. Lee, X. Zhang, and D. Xu, "LogGC: Garbage collecting audit log," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2013, pp. 1005–1016.
- [5] A. P. Chapman, H. V. Jagadish, and P. Ramanan, "Efficient provenance storage," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2008, pp. 993–1006.
- [6] W. Liwei, B. Zhifeng, H. Koehler, et al., "An approach for optimizing relational provenance storage," *Chinese J. Comput.*, vol. 34, no. 10, pp. 1863–1875, 2011.
- [7] Y. Xie, K.-K. Muniswamy-Reddy, D. D. Long, A. Amer, D. Feng, and Z. Tan, "Compressing provenance graphs," in *Proc. Workshop Theory Practice Provenance*, 2011, pp.1–5.
- [8] Y. Xie, K.-K. Muniswamy-Reddy, D. Feng, Y. Li, and D. D. Long, "Evaluation of a hybrid approach for efficient provenance storage," *ACM Trans. Storage*, vol. 9, no. 4, 2013, Art. no. 14.
- [9] Y. Xie, D. Feng, Z. Tan, L. Chen, K.-K. Muniswamy-Reddy, Y. Li, and D. D. Long, "A hybrid approach for efficient provenance storage," in *Proc. 21st ACM Int. Conf. Inf. Knowl. Manage.*, 2012, pp. 1752–1756.
- [10] Z. Xu, Z. Wu, Z. Li, K. Jee, J. Rhee, X. Xiao, F. Xu, H. Wang, and G. Jiang, "High fidelity data reduction for big data security dependency analyses," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2016, pp. 504–516.
- [11] K. H. Lee, X. Zhang, and D. Xu, "High accuracy attack provenance via binary-based execution partition," in *Proc. Annu. Netw. Distrib. Syst. Secur. Symp.*, 2013, pp.1–16.
- [12] P. Macko, D. Margo, and M. Seltzer, "Local clustering in provenance graphs," in *Proc. 22nd ACM Int. Conf. Inf. Knowl. Manage.*, 2013, pp. 835–840.

- [13] Z. Bao, H. Köhler, L. Wang, X. Zhou, and S. Sadiq, "Efficient provenance storage for relational queries," in *Proc. 21st ACM Int. Conf. Inf. Knowl. Manage.*, 2012, pp. 1352–1361.
- [14] K. K. Muniswamy-Reddy, D. A. Holland, U. Braun, and M. Seltzer, "Provenance-aware storage systems," in *Proc. Annu. Conf. USENIX Tech. Conf.*, 2006, pp. 43–56.
- [15] M. Seltzer, K. K. Muniswamy-Reddy, D. A. Holland, U. Braun, and J. Ledli, "Provenance-Aware Storage Systems," Harvard Univ., Cambridge, MA, USA, Tech. Rep. TR-18-05, 2005.
- [16] K. K. Muniswamy-Reddy and D. A. Holland, "Causality-based versioning," *ACM Trans. Storage*, vol. 5, no. 4, pp. 1–28, 2009.
- [17] K. K. Muniswamy-Reddy, U. Braun, D. A. Holland, P. Macko, D. Maclean, D. Margo, M. Seltzer, and R. Smogor, "Layering in provenance systems," in *Proc. Conf. USENIX Annu. Tech. Conf.*, 2009, pp. 10–10.
- [18] U. Braun, A. Shinnar, and M. Seltzer, "Securing provenance," in *Proc. Conf. Hot Topics Secur.*, 2008, Art. no. 4.
- [19] U. Braun and A. Shinnar, "A security model for provenance," Harvard Univ., Cambridge, MA, USA, Tech. Rep. TR-04-06, 2006.
- [20] R. Hasan, R. Sion, and M. Winslett, "The case of the fake Picasso: Preventing history forgery with secure provenance," in *Proc. Conf. File Storage Technol.*, 2009, pp. 1–14.
- [21] T. Gibson, K. Schuchardt, and E. Stephan, "Application of named graphs towards custom provenance views," in *Proc. 1st Workshop Theory Practice Provenance*, 2009, Art. no. 5.
- [22] C. Sar and P. Cao, "Lineage file system," 2005, pp. 411–414. [Online]. Available: <http://crypto.stanford.edu/cao/lineage.html>
- [23] A. Gehani and D. Tariq, "SPADE: Support for provenance auditing in distributed environments," in *Proc. 13th Int. Middleware Conf.*, 2012, pp. 101–120.
- [24] R. Spillane, R. Sears, C. Yalamanchili, S. Gaikwad, M. Chinni, and E. Zadok, "Story book: An efficient extensible provenance framework," in *Proc. 1st Workshop Theory Practice Provenance*, 2009, Art. no. 11.
- [25] A. Vahdat and T. E. Anderson, "Transparent result caching," in *Proc. Annu. Conf. USENIX Annu. Tech. Conf.*, 1998, pp. 3–3.
- [26] T. Pasquier, X. Han, M. Goldstein, T. Moyer, D. Eysers, M. Seltzer, and J. Bacon, "Practical whole-system provenance capture," in *Proc. Symp. Cloud Comput.*, 2017, pp. 405–418.
- [27] U. N. Raghavan, R. Albert, and S. Kumara, "Near linear time algorithm to detect community structures in large-scale networks," *Phys. Rev. E Statistical Nonlinear Soft Matter Physics*, vol. 76, 2007, Art. no. 036106.
- [28] Y. N. Xia, M. C. Zhou, X. Luo, S. C. Pang, and Q. S. Zhu, "Stochastic modeling and performance analysis of migration-enabled and error-prone clouds," *IEEE Trans. Ind. Informat.*, vol. 11, no. 2, pp. 495–504, Apr. 2015.
- [29] Z. Feng, Z. Feng, X. Wang, G. Rao, Y. Wei, and Z. Li, "HDStore: An SSD/HDD hybrid distributed storage scheme for large-scale data," in *Proc. Int. Conf. Web-Age Inf. Manage.*, 2014, pp. 209–220.
- [30] N. Agrawal, V. Prabhakaran, T. Wobber, J. D. Davis, M. Manasse, and R. Panigrahy, "Design tradeoffs for SSD performance," in *Proc. USENIX Tech. Conf.*, 2008, pp. 57–70.
- [31] D. Porcarelli, D. Brunelli, M. Magno, and L. Benini, "A multi-harvester architecture with hybrid storage devices and smart capabilities for low power systems," in *Proc. Int. Symp. Power Electron. Elect. Drives Autom. Motion*, 2012, pp. 946–951.
- [32] Y. Xie, D. Feng, Y. Hu, Y. Li, S. Sample, and D. D. Long, "Pagoda: A hybrid approach to enable efficient real-time provenance based intrusion detection in big data environments," *IEEE Trans. Depend. Secure Comput.*, 2018, doi: [10.1109/TDSC.2018.2867595](https://doi.org/10.1109/TDSC.2018.2867595).
- [33] 2015. [Online]. Available: <https://github.com/ashish-gehani/SPADE/wiki/ProvBench-Traces>
- [34] W. Xie, J. Zhou, M. Reyes, J. Noble, and Y. Chen, "Two-mode data distribution scheme for heterogeneous storage in data centers," in *Proc. IEEE Int. Conf. Big Data*, 2015, pp. 327–332.
- [35] B. Welch and G. Noer, "Optimizing a hybrid SSD/HDD HPC storage system based on file size distributions," in *Proc. IEEE 29th Symp. Mass Storage Syst. Technol.*, 2013, pp. 1–12.
- [36] D. Zhao and I. Raicu, "HyCache: A user-level caching middleware for distributed file systems," in *Proc. IEEE 27th Int. Conf. Parallel Distrib. Process. Symp. Workshops PhD Forum*, 2013, pp. 1997–2006.
- [37] X. Xu, C. Yang, and J. Shao, "Data replica placement mechanism for open heterogeneous storage systems," *Procedia Comput. Sci.*, vol. 109, pp. 18–25, 2017.
- [38] J. Zhou, W. Xie, Q. Gu, and Y. Chen, "Hierarchical consistent hashing for heterogeneous object-based storage," in *Proc. IEEE Trustcom/BigDataSE/ISPA*, 2016, pp. 1597–1604.



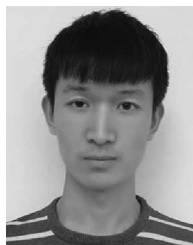
Die Hu received the BE degrees in computer science from Northeast Forestry University, China, in 2017. She is currently working toward the PhD degree at the Huazhong University of Science and Technology (HUST).



Dan Feng received the BE, ME, and PhD degrees in computer science and technology from the Huazhong University of Science and Technology (HUST), China, in 1991, 1994, and 1997, respectively. She is a professor and director of Data Storage System Division, Wuhan National Lab for Optoelectronics. She is also dean of the School of Computer Science and Technology, HUST. Her research interests include computer architecture, massive storage systems, parallel file systems, disk array, and solid state disk. She has more than 100 publications in journals and international conferences, including FAST, USENIX ATC, ICDCS, HPDC, SC, ICS and IPDPS. She is a member of the IEEE and ACM.



Yulai Xie received the BE and PhD degrees in computer science from the Huazhong University of Science and Technology (HUST), China, in 2007 and 2013, respectively. He was a visiting scholar with the University of California, Santa Cruz, in 2010 and a visiting scholar with the Chinese University of Hong Kong, in 2015. He is now an associate professor with the School of Computer Science and Technology, HUST, China. His research interests mainly include digital provenance, intrusion detection, network storage, and computer architecture. He is a member of the IEEE.



Gongming Xu received the BE degrees in computer science from the Wuhan Institute of Technology, China, in 2018. He is currently working toward the ME degree at the Huazhong University of Science and Technology (HUST).



Xinrui Gu received the BE and ME degrees in computer science from the Huazhong University of Science and Technology (HUST), China, in 2015 and 2018, respectively.



Darrell Long received the BS degree in computer science from San Diego State University, and the MS and PhD degrees from the University of California, San Diego. He is distinguished professor of computer engineering with the University of California, Santa Cruz. He holds the Kumar Malavalli endowed chair of Storage Systems Research and is director emeritus of the Storage Systems Research Center. His current research interests include the storage systems area include high performance storage systems, archival storage systems, and energy-efficient storage systems. His research also includes computer system reliability, video-on-demand, applied machine learning, mobile computing, and cyber security. He is a fellow of the IEEE and American Association for the Advancement of Science (AAAS).

▷ **For more information on this or any other computing topic, please visit our Digital Library at www.computer.org/csdl.**